Introduction

A village milk co-operative collects milk from farmers and pays them according to the quantity supplied. A simple payment calculator can calculate the amount due using the number of litres and the payment rate per litre. This assignment explains the problem, identifies the required data, explores Python values and data types, selects suitable variables and constants, and writes correct expressions to calculate the payment.

Assumptions

For this example, payment is calculated using a fixed rate per litre. The rate and quantity used in the example are illustrative values only; a real co-operative can replace them with its approved rate and measured quantity. The basic formula is: Total Payment = Quantity of Milk × Rate per Litre.

TASK 1 — Understand the Problem and Identify the Data

1. Problem statement

The co-operative needs a reliable way to calculate how much money should be paid to a farmer for the milk delivered. Manual calculations can lead to mistakes. A Python program can perform the multiplication consistently and display the payment clearly.

2. Input, process and output

Stage	Description
Input	Quantity of milk in litres and rate per litre in rupees.
Process	Multiply the quantity by the rate.
Output	Display the total payment in rupees, rounded to two decimal places.

1.3 Formula

```
total_payment = milk_quantity * rate_per_litre
```

Example: if a farmer supplies 12.5 litres and the rate is ■38.00 per litre, the payment is 12.5 × 38 = ■475.00.

TASK 2 — Explore Values and Types in the Interpreter

Python stores different kinds of values using data types. For this calculator, a quantity and a rate can be stored as floating-point numbers because they may contain decimal values. The calculated payment is also a float.

Example value	Python type	Purpose
12.5	float	Milk quantity in litres
38.0	float	Rate per litre
475.0	float	Calculated payment
"Farmer A"	str	Optional farmer name
True	bool	Example of a Boolean value

Interpreter examples

```
milk_quantity = 12.5
rate_per_litre = 38.0
total_payment = milk_quantity * rate_per_litre

print(type(milk_quantity))    # <class 'float'>
print(type(rate_per_litre))   # <class 'float'>
print(total_payment)          # 475.0
```

The type() function tells us the type of a value. The print() function displays a value on the screen.

TASK 3 — Choose Variables and Name the Constants

A variable is a named place used to store a value. Variable names should be meaningful and easy to understand. Python variable names are case-sensitive and cannot begin with a number. A constant is a value intended to remain unchanged while the program runs; Python commonly represents constants using uppercase names by convention.

Name	Kind	Meaning
MILK_QUANTITY	Constant (example)	A fixed sample quantity, if using a preset example.
RATE_PER_LITRE	Constant	Approved sample payment rate per litre.
milk_quantity	Variable	Quantity supplied by the farmer.
rate_per_litre	Variable	Rate used in the calculation.
total_payment	Variable	Calculated payment amount.

Naming example

```
RATE_PER_LITRE = 38.0
milk_quantity = 12.5
total_payment = milk_quantity * RATE_PER_LITRE
```

Here, RATE_PER_LITRE is written in uppercase because it is intended to act as a constant. The milk quantity can change for each farmer, so milk_quantity is a variable.

TASK 4 — Write Correct Expressions

Python expressions combine values, variables and operators to produce a result. The multiplication operator is * and the assignment operator is =. The round() function can round a result to two decimal places for display.

Complete Python program

```
# Payment calculator for a village milk co-operative

RATE_PER_LITRE = 38.0

milk_quantity = 12.5
total_payment = milk_quantity * RATE_PER_LITRE

print("Milk quantity:", milk_quantity, "litres")
print("Rate per litre: Rs.", RATE_PER_LITRE)
print("Total payment: Rs.", round(total_payment, 2))
```

Expected output

```
Milk quantity: 12.5 litres
Rate per litre: Rs. 38.0
Total payment: Rs. 475.0
```

Validation and test cases

Test	Quantity (litres)	Rate (■/litre)	Expected payment
1	12.5	38.0	■475.00

Test	Quantity (litres)	Rate (■/litre)	Expected payment
2	10.0	38.0	■ 380.00
3	0.0	38.0	■ 0.00

The test cases check a normal quantity, a whole-number quantity, and zero quantity. In an expanded program, the input should also be checked to ensure that the quantity is not negative and that the rate is valid.

Conclusion

This assignment demonstrates how Python can be used to calculate payments for a village milk co-operative. The required inputs and output were identified, suitable data types were explored, meaningful variable and constant names were selected, and a correct multiplication expression was implemented. The sample test cases confirm that the formula produces the expected payment. The same program can be adapted to use the co-operative's actual rate and measured milk quantity.